

PPCI 2025 - Maratona de Programação

5 de julho de 2025



Instruções Importantes

- Use a opção **Runs** para enviar suas soluções. Os problemas podem ser resolvidos em qualquer ordem e linguagem (dentro de C, C++ e Python, independentemente do problema);
- Suas soluções serão testadas com várias entradas, além das dadas como exemplo. Por isso, sua solução pode não ser aceita mesmo se funcionar para os exemplos dados. Certifique-se que ela funciona para todas as entradas possíveis;
- A saída gerada deve ser *exatamente* conforme especificada. Em particular, **não** imprima instruções (“digite um número”, “a resposta é”, etc);
- É garantido que todas as entradas usadas para teste estarão de acordo com o enunciado, não sendo necessário testar se são válidas;
- Ao enviar uma solução, o sistema irá responder uma das seguintes respostas:
 - **Not answered yet**: a solução está sendo corrigida. Aguarde um pouco e atualize a página;
 - **YES**: solução aceita. Parabéns!
 - **Wrong Answer**: a saída impressa pelo seu programa não é a saída correta esperada, para alguma entrada de teste;
 - **Presentation Error**: a saída impressa está correta, exceto por espaços em branco e/ou quebras-de-linha faltando/sobrando;
 - **Time Limit Exceeded**: o tempo de execução do seu programa ultrapassou o tempo limite estipulado para o problema (ver tabela abaixo). O tempo de execução da sua solução precisa ser menor;
 - **Runtime Error**: seu programa gerou algum erro em tempo de execução (“crashou”);
 - **Compile Error**: seu programa não compila.
- Todas as linhas, tanto na entrada quanto na saída, terminam com o caractere de fim-de-linha ($\backslash n$), mesmo quando houver apenas uma única linha na entrada e/ou saída;

- Sua solução deve processar cada arquivo de entrada no tempo máximo estipulado para cada problema, dado pela seguinte tabela:

Problema	Nome	Tempo Limite (segundos)
A	Agenda Cheia	1
B	Bashborne 2	1
C	Cifra de Eldoria	1
D	Defesa UFTPR	3
E	Enfileiramento Mágico	1
F	Fortaleza Subterrânea	2
G	Grande Sorteio Combinado	1
H	Harmonia do Amor	1
I	Incômodo	1
J	Joelho Ruim	2
K	Karma Palindrômico	1
L	Liane e o Hoverboard	1
M	Meryn	1
N	Novo Bloco	1
O	O Caixote	3
P	Pirâmide	1
Q	Quantos Pedros Tem em uma Linha Reta?	1

A: Agenda Cheia

Arquivo: agenda.[c|cpp|py]

Jorge contratou uma nova estagiária, a Carla, para trabalhar em sua empresa como secretária. Jorge estava muito ocupado, então ele pediu para Carla não marcar compromissos por enquanto. Mas como Carla é muito tímida, ela não teve coragem de dizer não aos clientes que ligaram para marcar compromissos.

Então ela teve uma brilhante ideia e marcou todos os compromissos para dia 31 de março, dia que ela acreditava fielmente que não existia. Mas para surpresa dela, março tem 31 dias e agora Jorge tem diversos compromissos marcados para o mesmo dia.

Jorge ficou muito bravo com Carla e mandou que ela organizasse a agenda de forma que ele conseguisse comparecer ao máximo de compromissos possíveis e que tinha 10 minutos para dizer quantos compromissos ele teria no dia, caso contrário estaria demitida. Carla está em pânico e pediu ajuda a você.

Entrada

A primeira linha da entrada contém o número N de compromissos marcados ($1 \leq N \leq 10^5$). Cada uma das N linhas seguintes contém 2 inteiros: I_i e F_i ($0 \leq I_i \leq 24, I_i \leq F_i \leq 24$), sendo, respectivamente, o horário de início e fim do compromisso i .

Saída

A saída deve conter um inteiro representando o número máximo de compromissos que Jorge pode comparecer.

Exemplo de entrada	Exemplo de saída
3 2 4 5 7 6 8	2

Exemplo de entrada	Exemplo de saída
5 14 23 2 4 6 11 17 21 6 6	4

Exemplo de entrada	Exemplo de saída
10 2 6 19 22 5 10 3 22 19 21 16 21 10 18 7 21 23 23 11 11	4

B: Bashborne 2

Arquivo: `bashborne2.[c|cpp|py]`

Um grupo de alunos iniciou uma produtora de software especializada no desenvolvimento de jogos eletrônicos, instalada em uma encubadora da UTFPR. Seu último lançamento se chama *Bashborne 2*, se trata de um *roguelitesoulslikebattleroyale* com elementos de extração. A cada partida, uma quantidade S de salas são geradas aleatoriamente, interconectadas. O jogador sempre inicia na sala 0, e extrai(encerra o jogo) após atravessar T salas.

Os desenvolvedores buscaram uma forma de medir o balanceamento de uma partida. Primeiro, eles atribuíram um valor de recompensa a cada sala, de acordo com seu conteúdo. Por segundo, calcularam a probabilidade do jogador escolher uma saída de acordo com a sala. Então, chegaram a seguinte fórmula:

$$Ganho_{s,t} = V_s + \sum_{(i,p) \in s} (p_i \cdot Ganho_{i,t-1})$$

Onde s e t são a sala e a quantia de passos, V é a recompensa da sala, i e p são a saída de uma sala e a probabilidade associada, respectivamente. A probabilidade das saídas de uma sala sempre soma 100%, as salas podem sempre ser revisitadas.

Entrada

A primeira linha contém dois inteiros S e T ($2 \leq S \leq 10^3$, $0 \leq T \leq 10^3$), o número de salas e a quantidade de passos. As salas são numeradas de 0 a $S - 1$.

A próxima linha contém S inteiros V_i ($-1000 \leq V_i \leq 1000$, $0 \leq i \leq S - 1$), sendo V_i a recompensa da sala de índice i .

As próximas S linhas descrevem as salas, as salas são descritas em ordem. Cada linha possui um inteiro E ($1 \leq E \leq 10$), a quantia de saídas da sala, seguido de E pares $s_i p_i$ ($0 \leq s_i \leq S - 1$, $1 \leq p_i \leq 100$), a sala de destino da saída e a probabilidade associada, em porcentagem.

Saída

Imprima o ganho calculado da partida, com precisão de 6 casas decimais, ciente que o jogador sempre inicia na sala 0.

Exemplo de entrada	Exemplo de saída
2 2 40 50 2 0 49 1 51 1 0 100	127.599000

Exemplo de entrada	Exemplo de saída
7 4 13 -517 -72 -93 248 -877 716 6 2 16 5 14 6 19 1 16 3 11 0 24 4 0 28 4 18 5 27 6 27 3 0 33 2 38 5 29 7 1 15 0 13 5 17 2 12 3 18 6 12 4 13 6 0 13 5 12 4 20 2 17 6 14 1 24 6 0 14 1 11 6 26 4 21 2 14 3 14 3 4 36 3 34 5 30	-338.156166

Exemplo de entrada	Exemplo de saída
3 0 300 469 -929 3 1 34 2 30 0 36 3 0 40 1 33 2 27 3 0 38 1 35 2 27	300.000000

C: Cifra de Eldoria

Arquivo: `cifra.[c|cpp|py]`

Você é Kael, o Guardião dos Arquivos Reais do reino de Eldoria. Uma mensagem secreta, contendo a localização de um artefato poderoso, precisa ser enviada para a general Lyra, que está no campo de batalha. Espiões do reino rival de Drakon estão por toda parte, e qualquer mensagem interceptada pode significar a ruína de Eldoria.

Felizmente, os antigos magos de Eldoria desenvolveram um método de criptografia "in-quebrável". O método funciona da seguinte maneira:

1. A mensagem secreta é convertida em um número inteiro, B .
2. Usa-se uma chave de criptografia pública, que consiste em dois números: E e M ;
3. A mensagem criptografada, é calculada pela fórmula: $(B^E) \bmod M$.

Para contornar essa limitação, os magos desenvolveram um método que acelera o cálculo de potências, garantindo que os números intermediários permaneçam gerenciáveis.

Entrada

A entrada consistirá em uma única linha contendo três números inteiros separados por espaços: B ($1 \leq B < M$), E ($0 \leq E \leq 10^{18}$) e M ($2 \leq M \leq 10^9$)

Saída

Imprima um único número inteiro, o valor da mensagem criptografada C .

Exemplo de entrada	Exemplo de saída
12 5 19	8

Exemplo de entrada	Exemplo de saída
7 20 100	1

Exemplo de entrada	Exemplo de saída
1234567 1000000000000000000 987654321	726832162

D: Defesa UTFPR

Arquivo: defesa.[c|cpp|py]

A **UTFPR** — **Universidade Fictícia Federal do Paraná** está sob ataque de sua arqui-inimiga, a **URFTPR** — **Universidade Rival Fictícia Federal do Paraná**, e precisa, urgentemente, organizar seu exército de **Super Soldados da Engenharia da Computação**, de forma com que as N salas do campus da UTFPR sejam protegidas.

No entanto, como é extremamente difícil se tornar um Super Soldado da Engenharia da Computação, o número disponível de soldados é limitado, mais especificamente, existem S Super Soldados. Por isso, para garantir a defesa total do amado campus da UTFPR, é necessário que cada sala seja protegida por **um, e apenas um, Super Soldado**. Essa é a única maneira de garantir a vitória contra a URFTPR.

Mesmo entre os incríveis Super Soldados, há uma variação de forças:

- A força de um Super Soldado define a **distância máxima** (em número de conexões) que ele consegue proteger a partir de sua **sala base**.
- Ou seja, todas as salas que estão a uma distância menor ou igual à sua força a partir da sala base ficam protegidas por aquele soldado.

Por exemplo:

Se existem três salas conectadas: E-101, E-102 e E-103, sendo E-101 conectada a E-102 e E-101 conectada a E-103, então:

- Um Super Soldado posicionado em E-101 com força 1 conseguirá proteger as salas E-101, E-102 e E-103.

Sabe-se que o campus possui C corredores de distância unitária, pelos quais é possível se locomover livremente em ambos os sentidos. É garantido que não há mais de um corredor conectando o mesmo par de salas.

Você, o melhor aluno da UTFPR, foi convocado pelo General Ricardo Oliveira para resolver o seguinte problema: **determinar se a maneira atual com que os Super Soldados estão posicionados é ótima ou não, considerando suas forças e as respectivas salas em que estão posicionados**.

Entrada

A primeira linha contém três inteiros N , C e S ($1 \leq N \leq 10^6$, $N-1 \leq C \leq \min(10^7, \frac{N \times (N-1)}{2})$, $1 \leq S \leq N$), representando respectivamente o número de salas, o número de corredores e o número de Super Soldados.

As próximas C linhas contêm dois inteiros A e B ($1 \leq A, B \leq N$), indicando que existe um corredor bidirecional entre as salas A e B .

As próximas S linhas contêm dois inteiros K e F ($1 \leq K \leq N$, $0 \leq F \leq 10^6$), indicando que há um Super Soldado posicionado na sala K com força F .

Saída

Para cada caso de teste, imprima uma única linha contendo **Sim** se todas as salas estão protegidas, ou **Nao** caso contrário.

Exemplo de entrada	Exemplo de saída
3 2 1 1 2 1 3 1 1	Sim

Exemplo de entrada	Exemplo de saída
4 2 2 1 2 2 3 1 1 4 0	Nao

E: Enfileiramento Mágico

Arquivo: `enfileiramento.[c|cpp|py]`

No longínquo reino de PPCI-Land, o lendário Banco Arcano de Tesouros atende dragões, elfos e demais criaturas quando precisam converter gemas em moedas de ouro. Para organizar o atendimento, o arquimago-tesoureiro Ricardo distribui a cada criatura uma senha mágica — um número inteiro positivo que representa o seu nível de prioridade: quanto maior a senha, mais urgente. Para uma melhor organização, o arquimago segue as seguintes orientações:

1. As criaturas se alinham exatamente na ordem de chegada.
2. À meia-noite, o arquimago lança o feitiço *Ordo Decrescendum*, que reorganiza toda a fila em ordem decrescente de prioridade.
3. Algumas criaturas possuem um amuleto anti-magia; elas não mudam de lugar mesmo sob o feitiço.
4. A cada novo dia de atendimento o processo se repete do zero, com um novo conjunto de criaturas.

Como você é um mago muito habilidoso, o arquimago Ricardo lhe dá uma missão muito importante, para cada dia, contar quantas criaturas permanecem na mesma posição depois que o feitiço é aplicado.

Entrada

A primeira linha contém um número inteiro ($1 \leq D \leq 1000$), representando os dias mágicos. A próxima linha contém um inteiro N ($1 \leq N \leq 1000$), o número de criaturas na fila. A próxima linha contém inteiros P_i ($1 \leq P_i \leq 10^9$) representando as senhas mágicas de cada criatura, na ordem de chegada.

Saída

Para cada dia mágico, imprima o número de criaturas que não mudaram de posição após a fila ser reordenada.

Exemplo de entrada	Exemplo de saída
2 5 100 80 90 30 40 4 10 20 30 40	1 0

Exemplo de entrada	Exemplo de saída
1 4 50 50 50 50	4

F: Fortaleza Subterrânea

Arquivo: `fortaleza.[c|cpp|py]`

Uma descoberta incrível foi feita durante as obras no campus: a UFTPR foi construída em cima de um *bunker* subterrâneo abandonado! Pelo o que tudo indica, tratava-se de um *bunker* construído pela rival URTRP para espionar todas as tramas da futura universidade que seria construída ali.

Foram encontrados N computadores antigos, onde cada computador está fixado em uma posição (x_i, y_i) do *bunker*. Dos N computadores, S deles são *supercomputadores*, enquanto os demais são computadores comuns.

Se os S supercomputadores forem interligados em uma rede, algo muito grande irá acontecer (não sabemos ainda o que)! Por isso, você irá conectar pares de computadores com cabos de rede de forma que os *supercomputadores* possam se comunicar entre si, seja diretamente ou através de outros computadores (super ou comuns). Note que não é necessário que todos os computadores se comuniquem, mas apenas os supercomputadores.

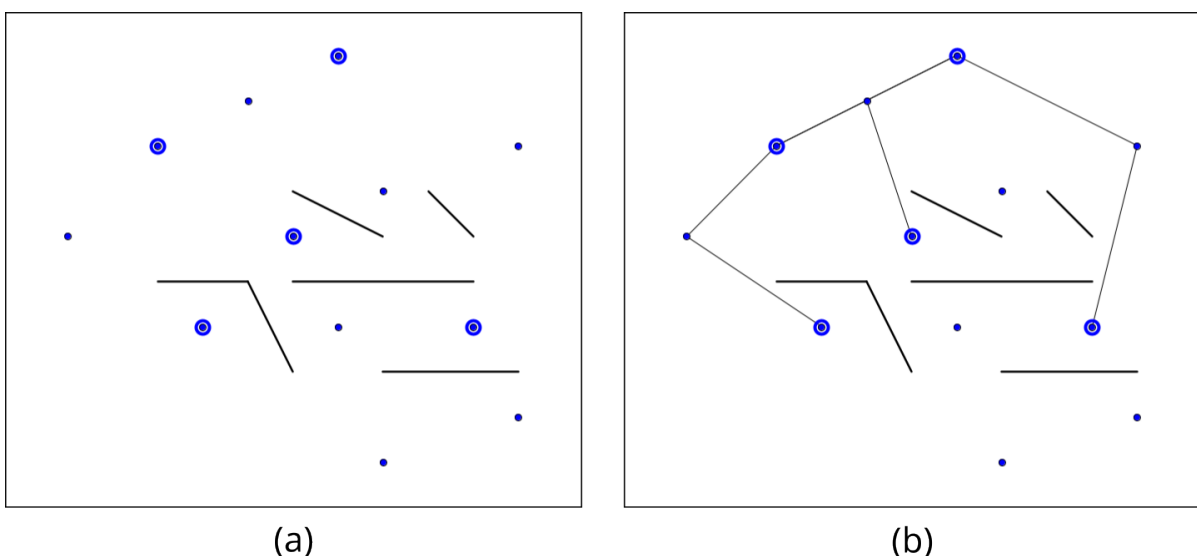
Entretanto, há um problema: por causa das obras no campus, M muros foram fixados no solo, de forma que é impossível passar um cabo de rede por um muro. Os cabos de rede devem ser instalados de forma que:

- o cabo que liga dois computadores deve ser *reto* (o cabo não pode fazer curvas nem desvios);
- o cabo não pode passar por um muro (nem mesmo por uma extremidade do muro);
- nenhum outro computador pode ser incluído à rede.

O custo da instalação de um cabo de rede é igual à distância entre os dois computadores que ele liga.

É possível interligar todos os supercomputadores com as restrições dadas? Se sim, qual é o custo total mínimo necessário para tal?

A figura abaixo apresenta (a) o primeiro exemplo de entrada e (b) uma solução de custo total mínimo para ele.



Entrada

A primeira linha contém dois inteiros N e M ($2 \leq N \leq 50$, $0 \leq M \leq 50$), o número de computadores e de muros. Os computadores são numerados de 1 a N .

As próximas N linhas descrevem os computadores. A linha i contém dois inteiros x_i, y_i ($0 \leq x_i, y_i \leq 10^5$) indicando as coordenadas do computador i .

As próximas M linhas descrevem os muros. A linha i contém quatro inteiros x_1, y_1, x_2, y_2 ($0 \leq x_1, y_1, x_2, y_2 \leq 10^5$, $(x_1, y_1) \neq (x_2, y_2)$) indicando um muro de extremidades (x_1, y_1) e (x_2, y_2) .

A próxima linha contém um inteiro S ($2 \leq S \leq \min(N, 8)$) indicando o número de supercomputadores. A linha seguinte contém S inteiros distintos s ($1 \leq s \leq N$), indicando que o computador s é um supercomputador.

É garantido que não há dois computadores no mesmo ponto, e que nenhum computador está exatamente abaixo de um muro.

Saída

Imprima o custo total mínimo necessário para interligar os supercomputadores, com três casas decimais. Caso não seja possível interligá-los, imprima `impossivel`.

Exemplo de entrada	Exemplo de saída
12 6 7 1 3 4 0 6 6 4 10 2 9 4 4 9 7 7 10 8 6 10 5 6 2 8 4 5 2 5 5 7 7 6 7 3 10 3 9 6 8 7 5 3 4 5 5 5 9 5 5 2 6 10 11 12	22.664

Exemplo de entrada	Exemplo de saída
2 0 0 0 1 1 2 1 2	1.414

Exemplo de entrada	Exemplo de saída
4 1 2 4 6 4 4 2 4 6 4 3 4 5 2 1 2	5.657

Exemplo de entrada	Exemplo de saída
4 1 2 4 6 4 4 2 4 6 3 5 5 3 2 1 2	impossivel

Exemplo de entrada	Exemplo de saída
4 1 2 4 4 3 7 5 4 6 3 5 6 5 2 1 4	8.261

G: Grande Sorteio Combinado

Arquivo: `sorteio.[c|cpp|py]`

Imagine que estejam promovendo um sorteio em que os números a serem sorteados são formados pelas combinações de N , P a P . Ou seja, cada número do sorteio é formado pela escolha de P números dentre os números de 1 a N , inclusive.

Porém, ao anunciar o sorteio, descobriu-se muitos interessados. De forma que seria demasiadamente trabalhoso para a organização enumerar todas as combinações para vendê-las aos participantes e para realizar o sorteio em si.

Além disso, pela sua popularidade inesperada, deseja-se promovê-lo novamente, com mais ou menos números, dependendo do sucesso desse evento.

Sendo assim, você ficou responsável por criar um programa que, dados os valores de N e P , gere as combinações para venda e, realizado o sorteio, indique a combinação vencedora. Para simplificar o programa, você decidiu gerar as combinações em ordem lexicográfica, de modo que:

- na combinação, os números estarão em ordem decrescente (mesmo que combinação não leve em conta a ordem dos elementos);
- as combinações serão geradas em ordem crescente.

Por exemplo, para $N = 5$ e $P = 3$, as combinações seriam geradas na seguinte ordem:

3	2	1
4	2	1
4	3	1
4	3	2
5	2	1
5	3	1
5	3	2
5	4	1
5	4	2
5	4	3

Perceba que, como as combinações foram geradas de forma ordenada e podem ser enumeradas, o problema de sortear uma dentre todas elas pode ser resolvido sorteando um número K entre 1 e C_P^N , inclusive. Isso seria o equivalente a sortear a K –ésima combinação.

Para o exemplo acima, se o número sorteado fosse $K = 4$, a combinação vencedora seria (4, 3, 2).

Entrada

Uma única linha contendo três inteiros N , P e K ($1 \leq P \leq N \leq 25$), ($1 \leq K \leq C_P^N$).

Saída

Uma única linha contendo a K –ésima combinação de N , P a P , conforme os critérios definidos no enunciado e no formato apresentado no exemplo. Com os elementos separados por um único espaço entre si e quebra de linha no final.

Exemplo de entrada 5 3 4	Exemplo de saída 4 3 2
------------------------------------	----------------------------------

Exemplo de entrada 6 2 7	Exemplo de saída 5 1
------------------------------------	--------------------------------

Exemplo de entrada 7 4 21	Exemplo de saída 7 5 3 1
-------------------------------------	------------------------------------

Exemplo de entrada 18 7 23757	Exemplo de saída 18 15 14 7 4 3 2
---	---

H: Harmonia do Amor

Arquivo: `harmonia.[c|cpp|py]`

Doctor Love está desenvolvendo um novo aplicativo de namoro revolucionário. Segundo ele, o segredo do amor está... na matemática! A ideia é simples: o usuário digita o nome de duas pessoas interessadas. O programa deve contar quantas letras tem cada nome e, em seguida, somar a quantidade de letras do primeiro nome com a do segundo. Se o a soma da quantidade de letras for um número par, então temos um MATCH! Caso contrário, é NO MATCH.

Entrada

Duas linhas, cada uma contendo um nome (apenas letras, sem espaços, com até 100 caracteres).

Saída

Se for um match segundo o método de Doctor Love, imprima "MATCH" (sem as aspas). Caso contrário, imprima "NO MATCH" (também sem as aspas).

Exemplo de entrada	Exemplo de saída
ana bruno	MATCH

Exemplo de entrada	Exemplo de saída
kaleb fernanda	NO MATCH

Exemplo de entrada	Exemplo de saída
carla igor	NO MATCH

I: Incômodo

Arquivo: `incomodo.[c|cpp|py]`

Um aluno da **UFTPR** (Universidade Fictícia Tecnológica do Paraná) está enfrentando um pequeno incômodo: os nomes das disciplinas em seu horário de aulas são longos demais e ocupam muito espaço em suas anotações e planilhas. Para facilitar sua organização, ele decidiu criar um programa que abrevie automaticamente os nomes das matérias, mantendo apenas as letras iniciais de cada palavra.

Ajude nosso caro aluno a implementar essa funcionalidade em código. O objetivo é ler o nome completo de uma disciplina e gerar sua abreviação. Por exemplo, "Cálculo Diferencial Integral 1" deve ser abreviado como "CDI1"

Entrada

A primeira linha contém um inteiro N ($1 \leq N \leq 1000$), representando o número de strings que serão lidas.

As próximas N linhas contêm, cada uma, uma cadeia de caracteres (string), contendo apenas letras (maiúsculas ou minúsculas), com espaços.

Saída

Para cada string de entrada, imprima em uma linha com letras maiúsculas a sigla de cada matéria:

Exemplo de entrada	Exemplo de saída
3 Calculo Diferencial Integral 3 Equacoes Diferenciais Ordinarias Geometria Analitica	CDI3 EDO GA

Exemplo de entrada	Exemplo de saída
2 Algoritmo Estrutura Dados 1 Circuitos eletricos 1	AED1 CE1

Exemplo de entrada	Exemplo de saída
3 Equacoes Diferenciais Parciais Circuitos Eletricos 2 Calculo Diferencial Integral 1	EDP CE2 CDI1

J: Joelho Ruim

Arquivo: joelho.[c|cpp|py]

Douglinhas era um excelente estudante da UTFPR: estudioso, tirava boas notas, participava da equipe de maratona de programação e estava sempre disposto a aprender mais. Para tornar sua vida acadêmica ainda mais dinâmica, decidiu começar a treinar basquete pela atlética da sua universidade. No entanto, em um dos treinos, ao realizar um movimento errado, Douglinhas acabou rompendo seu joelho, o que o forçou a interromper sua carreira como atleta.

Mesmo assim, Douglinhas continuou frequentando os treinos, mas como não podia mais jogar, passou a acompanhar seus colegas apenas do banco. Foi então que percebeu como era tedioso ficar sentado, sem participar. No meio desse tédio, sua mente de programador começou a divagar, até que surgiu uma pergunta curiosa:

“Se eu tivesse uma string de tamanho N , como eu poderia escrever um programa que me retornasse a maior substring palindrômica que se repete ao menos duas vezes na string, garantindo que essas repetições não sejam sobrepostas?”

Intrigado, Douglinhas foi para casa determinado a resolver esse problema. No entanto, após várias tentativas sem sucesso, ele percebeu que precisava de ajuda.

Entrada

Uma linha contendo a string S composta apenas por letras minúsculas, com $1 \leq |S| \leq 10^5$.

Saída

Imprima a maior substring palindrômica que tem repetição sem sobreposição em S . Caso existe múltiplas substrings como resposta certa, imprima a lexicograficamente menor. Se não houver nenhuma substring como resposta, imprima -1.

É garantido que a resposta, quando existir, não terá mais que 50 caracteres.

Exemplo de entrada	Exemplo de saída
abc	-1

Exemplo de entrada	Exemplo de saída
abccba	a

Exemplo de entrada	Exemplo de saída
abccbadabccba	abccba

K: Karma Palindrômico

Arquivo: `karma.[c|cpp|py]`

A **UFTPR** (Universidade Fictícia Tecnológica do Paraná) possui um vasto acervo de títulos literários em sua biblioteca. Recentemente, um grupo de estudos de linguística computacional propôs identificar em um livro sobre cultura oriental certas palavras, mais especificamente aquelas que são *palíndromas* - palavras ou frases que podem ser lidas da mesma forma de trás para frente.

Para facilitar essa análise, os alunos precisam desenvolver um programa que recebe múltiplas cadeias de caracteres e, para cada uma, determina se ela é palíndroma. Você conseguiria ajudá-los?

Dada uma sequência de N cadeias de caracteres não vazias, cada uma com até 1000 caracteres (somente letras maiúsculas e minúsculas, sem espaços), determine para cada cadeia se ela é um palíndromo.

Entrada

A primeira linha contém um inteiro N ($1 \leq N \leq 100000$), representando o número de strings que serão lidas.

As próximas N linhas contêm, cada uma, uma cadeia de caracteres (string), contendo apenas letras (maiúsculas ou minúsculas), sem espaços.

Saída

Para cada string de entrada, imprima em uma linha:

- **SIM** se a cadeia for palíndroma;
- **NAO** caso contrário.

Exemplos

Exemplo de entrada	Exemplo de saída
3 abcdedcba asdjgkgjdsa palindroma	SIM SIM NAO

L: Liane e o Hoverboard

Arquivo: `liane.[c|cpp|py]`

Um invasor de outro mundo está destruindo a cidade.

A equipe de resposta a incidentes como esse lida com o invasor, mas com dificuldades.

Liane, a engenheira da equipe, precisa ajudá-los, e ela sabe como. Com muito esforço, ela conseguiu, durante a invasão, terminar a implementação de um dispositivo capaz de retroceder o tempo, que pode ser muito útil em batalha.

Entretanto, há um problema: o local ao qual ela precisa ir é distante, e Liane não possui nenhum meio de locomoção que não seu hoverboard flutuante elétrico, que perde 62,5% de bateria a cada quilômetro percorrido. Felizmente, ao longo do caminho, existem estações de recarga suficientemente perto uma das outras para que ela consiga chegar ao local. Além disso, o hoverboard está completamente carregado.

Infelizmente, a cada parada em uma estação, Liane precisa esperar 60 segundos para recarregar o hoverboard, esteja ele completamente descarregado ou não. O tempo é crítico nesta situação. Ela precisa ajudar sua equipe o mais rápido possível.

Seja E o conjunto das estações de recarga presentes no caminho que Liane precisa fazer, D o conjunto de distâncias dessas estações e f a distância do local em que a equipe de Liane está. Todas as distâncias são relativas ao ponto de partida inicial de Liane, na base da equipe, são números inteiros, e são dadas em metros.

Determine por quais estações Liane deve passar de forma que ela perca a menor quantidade de tempo possível recarregando seu hoverboard elétrico, e assim consiga ajudar sua equipe o quanto antes.

Entrada

A primeira linha é um inteiro N ($1 < N \leq 10^5$), que representa o número de estações no caminho.

Cada uma das seguintes N linhas contém dois inteiros: o número da estação e_i ($0 \leq e_i \leq 10^5$), seguido de sua distância d_i ($d_i \leq f$). A última linha é a distância f ($2000 \leq f \leq 1,600016 \cdot 10^8$) do local em que a equipe de Liane está.

Saída

A saída deve conter a sequência de estações pelas quais Liane deve passar, de forma que minimize o tempo gasto para recarregar seu hoverboard.

Exemplo de entrada	Exemplo de saída
17 64 4140 35 5080 55 8166 9 6905 83 11714 5 5639 46 3167 61 10628 56 3750 79 4505 42 604 15 9367 7 13269 51 13845 60 1928 86 10092 43 6179 14125	42 60 46 79 5 9 55 15 61 83 7

Exemplo de entrada	Exemplo de saída
7 80 3966 82 1234 5 2082 51 6264 81 5375 36 2868 75 2385 6292	82 75 80 81

Exemplo de entrada	Exemplo de saída
4 6 1782 18 1571 72 919 94 450 2200	18

M: Meryn

Arquivo: `meryn.[c|cpp|py]`

Após quase uma década de árduas construções atravessando montanhas antigas, bosques encantados e planícies esquecidas pelos mapas imperiais, as tão aguardadas ferrovias da lendária Terra de Meryn finalmente chegaram à conclusão. Aclamadas como o maior feito de engenharia dos últimos séculos, elas provocaram um crescimento assombroso no Produto Interno Bruto — um salto de 58%, com margem de erro estimada em 2%, segundo os eruditos da Guilda Econômica Real. Essas vias férreas, projetadas por engenheiros-magos da Ordem de Setealém, têm uma composição bastante peculiar:

O conjunto de vias implantado, deu origem a uma malha ferroviária que pode ser percorrida continuamente de uma extremidade à outra, sem revisitar paradas nem cruzar de forma concorrente qualquer outra linha.

Kindle Bilheterias surgem e desaparecem com notável frequência, como se obedecessem a um ritmo próprio ditado pelos ventos do comércio e pelas guerras nas fronteiras. Diante desse cenário mutável, o Alto Conselho de Transportes de Meryn te convocou para desenvolver um algoritmo confiável, que será integrado ao sistema central de controle das passagens, conhecido como Olho d3 DanTe. Sua missão é implementar uma solução que, dada a quantidade de bilheterias ativas na malha férrea, determine quantos tipos distintos de bilhetes devem ser criados. Cada bilhete conterá as seguintes informações:

Origem Destino Nome completo do passageiro Registro Patriótico (RP), um código identificador único fornecido pelo Reino.

Contudo, para fins de sistema e logística, um “tipo de bilhete” é definido apenas pelo par origem-destino — ou seja, bilhetes com a mesma origem e destino (independentemente do passageiro ou do RP) são considerados do mesmo tipo.

Entrada

A entrada consiste em um inteiro positivo A , representando a quantidade de Kindle Bilheterias existentes na malha férrea.

Saída

Seu programa deve retornar a quantidade total de tipos distintos de bilhetes que deverão ser desenvolvidos, considerando todas as possíveis combinações de origem e destino.

Exemplo de entrada	Exemplo de saída
12	132

Exemplo de entrada	Exemplo de saída
19	342

Exemplo de entrada	Exemplo de saída
14	182

N: Novo Bloco

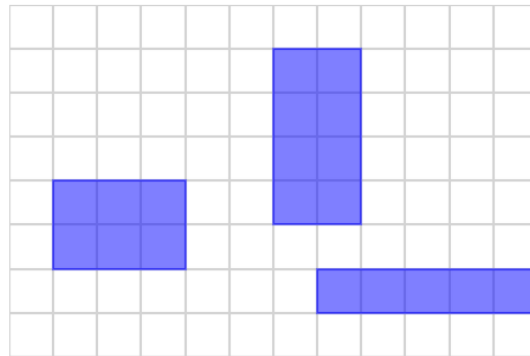
Arquivo: novo.[c|cpp|py]

A UFTPR tem um terreno vazio em seu campus e deseja contruir um novo bloco nele. Por motivos técnicos, o terreno é dividido em formato de uma grid retangular com a linhas e b colunas, e toda construção no terreno deve seguir essa grid. A equipe de design do campus decidiu que o novo bloco deve ter um formato retangular, mas que não seja quadrado.

A universidade pediu a sua ajuda no departamento de computação para saber quantas formas existem para construir um bloco nessas condições.

Exemplo

Abaixo há um exemplo de alguns possíveis blocos que podem ser construidos em um terreno de tamanho 8x12.



Entrada

A entrada é composta de apenas dois valores inteiros a e b ($1 \leq a, b \leq 10^9$), onde a indica o numero de linhas da grid e b indica o número de colunas.

Saída

Imprima o número de formas de se construir um bloco nas condições dadas, módulo 998244353.

Exemplo de entrada	Exemplo de saída
2 4	19

Exemplo de entrada	Exemplo de saída
5 5	170

Exemplo de entrada	Exemplo de saída
7 8	840

O: O Caixote

Arquivo: ocaixote.[c|cpp|py]

A Universidade Formadora de Transformação de Programas e Recursos (UTFPR) está em busca de programadores criativos para resolver um mistério em seu galpão abandonado! Recentemente, descobrimos que o galpão está cheio de objetos estranhos e precisamos de caixas de madeira personalizadas para organizá-los. Mas não são caixas comuns — elas devem ser tão únicas quanto os objetos que guardarão! Sua tarefa é fazer um programa com um algoritmo que imprime o tamanho das caixas de acordo com a escolha do usuário para assim serem construídas. Lembre-se que a caixa tem duas paletas diagonais de sustentação, em formato de 'X'.

Entrada

Um único inteiro N ($5 \leq N \leq 100$), representando o tamanho da caixa (largura e altura).

Saída

Imprime uma caixa de madeira no terminal usando asteriscos '*' de acordo com inteiro inserido na entrada.

Exemplo de entrada	Exemplo de saída
5	<pre>***** ** ** * * * ** ** *****</pre>
10	<pre>***** ** ** * * * * * * * * * ** * * ** * * * * * * * * * ** ** *****</pre>

P: Pirâmide

Arquivo: `piramide.[c|cpp|py]`

Construa um algoritmo que apresenta na tela uma pirâmide de base A, com o caractere asterisco ”*”. Entre dois asteriscos sempre há um espaço.

Entrada

A entrada é composta por um inteiro, que representa o tamanho da base A.

Saída

A pirâmide.

Exemplo de entrada	Exemplo de saída
1	*

Exemplo de entrada	Exemplo de saída
5	<pre> * * * * * * * * * * * * * * *</pre>

Exemplo de entrada	Exemplo de saída
12	<pre> * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *</pre>

Q: Quantos Pedros tem em uma Linha Reta?

Arquivo: `quantos.[c|cpp|py]`

Luquinhas é um excelente estudante da UTFPR e membro da equipe de maratona de programação. Em 2024, Luquinhas e sua equipe se classificaram para a Final Brasileira em Jon Person. No entanto, devido à distância entre sua cidade, Toledo, e Jon Person, eles precisaram pegar vários voos com conexões, totalizando 6 aeroportos entre ida e volta.

Durante essas longas esperas em aeroportos, Luquinhas desenvolveu a habilidade incrível de dormir em qualquer banco, não importa o quão desconfortável seja. Em uma dessas sonecas, sua mente inquieta de maratonista teve um sonho um tanto curioso.

No sonho, Luquinhas imaginou um mundo plano (afinal, quem liga pra geografia no mundo dos sonhos?) onde existem várias concentrações populacionais representadas por círculos em um plano cartesiano. No meio dessa loucura, ele se perguntou:

“Pedro é um nome um tanto comum. Se eu traçar um segmento de reta nesse mundo plano, quantos Pedros será que essa linha vai cruzar?”

Como ele também é uma pessoa lógica, seu sonho incluiu uma fórmula para calcular a “probabilidade de cruzar com um Pedro”: a diferença entre o raio de uma concentração populacional e a distância do segmento de reta ao centro do círculo (quanto mais perto do centro, maior a chance de encontrar um Pedro!).

Sua tarefa é ajudar Luquinhas a calcular a soma dessas probabilidades para todas as concentrações populacionais que forem intersectadas pelo segmento de reta.

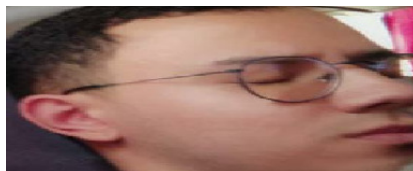
Entrada

A entrada é composta por uma única linha com cinco números reais: x_1, y_1, x_2, y_2 ($-10^4 \leq x_1, y_1, x_2, y_2 \leq 10^4$) representando as coordenadas do ponto inicial do segmento de reta e coordenadas do ponto final do segmento de reta respectivamente e um número N ($1 \leq N \leq 10^4$) representando a quantidade de círculos.

As próximas N linhas contêm, cada uma, três números reais: x, y ($-10^4 \leq x, y \leq 10^4$) as coordenadas do centro do círculo e r ($0 < r \leq 10^3$) representando o raio de cada círculo.

Saída

Imprima um único número real com 4 casas decimais representando o somatório das probabilidades de se ter um Pedro de cada concentração populacional.



Exemplo de entrada	Exemplo de saída
0 0 3 3 1 1 1 2	2.0000

Exemplo de entrada	Exemplo de saída
0 0 10 0 2 5 0.42 2 5 5 2	1.5800

Exemplo de entrada	Exemplo de saída
-5 -5 5 5 2 0 0 1 10 10 5	1.0000